

```
# DSA03 - Natural Language Processing
# CO3 - Assessment Tool 1

# Experiment 1: CFG Rule Validation and Parse Tree Construction
# Experiment 2: Agreement and Feature Structure Checking
# Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

# Name: P. Tejasri
# Reg. No.: 192472069
```

```
# Experiment 1: CFG Rule Validation and Parse Tree Construction
```

```
# Define the grammar
grammar = {
    "Det": ["the", "a"],
    "N": ["dog", "cat", "bone"],
    "V": ["chased", "found"]
}

# Function to parse and validate a sentence
def parse_cfg(sentence):

    # Convert sentence into lowercase words
    words = sentence.lower().split()

    # According to the grammar:
    # S -> NP VP
    # NP -> Det N
    # VP -> V NP
    # Therefore, the sentence must contain 5 words.
    if len(words) != 5:
        return "Invalid Sentence"

    det1, noun1, verb, det2, noun2 = words

    # Validate first NP
    if det1 not in grammar["Det"]:
        return "Invalid Sentence"

    if noun1 not in grammar["N"]:
        return "Invalid Sentence"

    # Validate V
    if verb not in grammar["V"]:
        return "Invalid Sentence"

    # Validate second NP
    if det2 not in grammar["Det"]:
        return "Invalid Sentence"

    if noun2 not in grammar["N"]:
        return "Invalid Sentence"

    # Construct parse tree
    parse_tree = f"""
S
├── NP
│   ├── Det → {det1}
│   └── N → {noun1}
└── VP
    ├── V → {verb}
    └── NP
        ├── Det → {det2}
        └── N → {noun2}
"""

    return "Valid Parse Tree:\n" + parse_tree

# Test cases given in the assessment
test_sentences = [
    "the dog chased a cat",
    "a cat found the bone",
    "dog chased cat",
    "the bone found a dog",
    "chased the dog bone"
]
```

```
# Display results
for sentence in test_sentences:
    print("Sentence:", sentence)
    print(parse_cfg(sentence))
    print("-" * 50)
```

Sentence: the dog chased a cat
Valid Parse Tree:

```

S
├── NP
│   ├── Det → the
│   └── N → dog
└── VP
    ├── V → chased
    └── NP
        ├── Det → a
        └── N → cat

```

Sentence: a cat found the bone
Valid Parse Tree:

```

S
├── NP
│   ├── Det → a
│   └── N → cat
└── VP
    ├── V → found
    └── NP
        ├── Det → the
        └── N → bone

```

Sentence: dog chased cat
Invalid Sentence

Sentence: the bone found a dog
Valid Parse Tree:

```

S
├── NP
│   ├── Det → the
│   └── N → bone
└── VP
    ├── V → found
    └── NP
        ├── Det → a
        └── N → dog

```

Sentence: chased the dog bone
Invalid Sentence

Experiment 2: Agreement and Feature Structure Checking

Feature structures for subjects

```
subject_features = {
    "he": {
        "number": "singular",
        "person": "third"
    },
    "she": {
        "number": "singular",
        "person": "third"
    },
    "it": {
        "number": "singular",
        "person": "third"
    },
    "we": {
        "number": "plural",
        "person": "first"
    }
}
```

Feature structures for verbs

```
verb_features = {
    "eats": {
        "number": "singular"
```

```

    },

    "sleeps": {
        "number": "singular"
    },

    "eat": {
        "number": "plural"
    },

    "sleep": {
        "number": "plural"
    }
}

# Function to check subject-verb agreement
def check_agreement(subject, verb):

    # Convert input to lowercase
    subject = subject.lower()
    verb = verb.lower()

    # Check whether subject exists
    if subject not in subject_features:
        return False

    # Check whether verb exists
    if verb not in verb_features:
        return False

    # Get subject number
    subject_number = subject_features[subject]["number"]

    # Get verb number
    verb_number = verb_features[verb]["number"]

    # Agreement holds when the numbers are the same
    if subject_number == verb_number:
        return True
    else:
        return False

# Test cases given in the assessment
test_cases = [
    ("he", "eats"),
    ("he", "eat"),
    ("we", "eat"),
    ("we", "sleeps"),
    ("she", "sleeps")
]

# Display results
for subject, verb in test_cases:

    result = check_agreement(subject, verb)

    print("Subject:", subject)
    print("Verb:", verb)
    print("Agreement:", result)
    print("-" * 40)

```

```

Subject: he
Verb: eats
Agreement: True
-----
Subject: he
Verb: eat
Agreement: False
-----
Subject: we
Verb: eat
Agreement: True
-----
Subject: we
Verb: sleeps
Agreement: False
-----
Subject: she
Verb: sleeps
Agreement: True
-----

```

```
# Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing
```

```
# Probability of S -> NP VP
```

```
S_probability = 1.0
```

```
# Probabilities of NP rules
```

```
NP_probabilities = {
```

```
    "Tom": 0.7,
```

```
    "Ann": 0.3
```

```
}
```

```
# Probabilities of VP rules
```

```
VP_probabilities = {
```

```
    "sings": 0.4,
```

```
    "dances": 0.6
```

```
}
```

```
# Function to calculate sentence probability
```

```
def sentence_probability(sentence):
```

```
    # Split the sentence into words
```

```
    words = sentence.split()
```

```
    # The grammar generates a two-word sentence
```

```
    if len(words) != 2:
```

```
        return 0.0
```

```
    subject = words[0]
```

```
    verb = words[1]
```

```
    # Check whether subject can be generated by NP
```

```
    if subject not in NP_probabilities:
```

```
        return 0.0
```

```
    # Check whether verb can be generated by VP
```

```
    if verb not in VP_probabilities:
```

```
        return 0.0
```

```
    # Get NP probability
```

```
    np_probability = NP_probabilities[subject]
```

```
    # Get VP probability
```

```
    vp_probability = VP_probabilities[verb]
```

```
    # Calculate:
```

```
    # P(Sentence) = P(S -> NP VP) * P(NP) * P(VP)
```

```
    probability = (
```

```
        S_probability *
```

```
        np_probability *
```

```
        vp_probability
```

```
)
```

```
    return probability
```

```
# Test cases given in the assessment
```

```
test_sentences = [
```

```
    "Tom sings",
```

```
    "Tom dances",
```

```
    "Ann sings",
```

```
    "Ann dances",
```

```
    "Sam sings"
```

```
]
```

```
# Display sentence probabilities
```

```
for sentence in test_sentences:
```

```
    probability = sentence_probability(sentence)
```

```
    print(
```

```
        "Sentence:",
```

```
        sentence,
```

```
        "→ Probability:",
```

```
        f"{probability:.2f}"
```

```
)
```

```
Sentence: Tom sings → Probability: 0.28
```

```
Sentence: Tom dances → Probability: 0.42
```

```
Sentence: Ann sings → Probability: 0.12
```

```
Sentence: Ann dances → Probability: 0.18  
Sentence: Sam sings → Probability: 0.00
```